

PIXEL SOFTWARE DESIGN

Département de Recherche & Développement

Intelligence Artificielle Avancée

& Federated Learning

- Fondations de l'intelligence artificielle
- Algorithmes d'apprentissage fédéré
- Confidentialité, attaques et défenses
- Applications et grands modèles fédérés
- Framework FedX de référence
- Nouveaux fronts de recherche
- Travaux pratiques, projets et évaluation

Auteur	**Ghiloufi Jamel**
Fonction	**Architecte Logiciel Principal**
Direction	**Département de Recherche & Développement**
Service	**R&D - Intelligence Artificielle Avancée & Apprentissage Fédéré**
Niveau	**Recherche**
Édition	**Édition 2026 – 1.0**
Copyright	**© Pixel Software Design**

Ce document est la propriété exclusive de Pixel Software Design. Toute reproduction, représentation, adaptation ou diffusion, totale ou partielle, par quelque procédé que ce soit et sur quelque support que ce soit, est interdite sans autorisation écrite préalable de l'auteur.

Copyright et mentions légales

Éditeur	Pixel Software Design
Auteur	Ghiloufi Jamel
Fonction	Architecte Logiciel Principal
Département	Département de Recherche & Développement
Intitulé	Intelligence Artificielle Avancée & Federated Learning
Sous-titre	Fondations mathématiques de l'apprentissage automatique, algorithmes fédérés, protection de la vie privée, défense contre les attaques, grands modèles de langage fédérés, systèmes embarqués, renforcement appris, applications critiques (santé, V2X), plateforme de référence FedX et feuille de route de recherche.
Édition	Édition 2026 (version 1.0)
Langue	Français
Niveau	Recherche
Collection	Supports de formation et de recherche, Pixel Software Design

Droits d'auteur

© 2026 Pixel Software Design. Tous droits réservés.

Ce document est la propriété exclusive de Pixel Software Design. Toute reproduction, représentation, adaptation ou diffusion, totale ou partielle, par quelque procédé que ce soit et sur quelque support que ce soit, est interdite sans autorisation écrite préalable de l'auteur.

Droits de l'auteur. Ghiloufi Jamel est l'auteur unique de l'ensemble du contenu scientifique, pédagogique et technique de cet ouvrage. Toute adaptation, traduction, reproduction ou exploitation, y compris partielle, requiert une autorisation écrite préalable.

Droits des tiers. Les algorithmes, modèles et travaux cités restent la propriété de leurs auteurs respectifs. Les références employées figurent en fin d'ouvrage, ce qui permet de remonter au travail original pour toute affirmation chiffrée.

Marques et licences externes. PyTorch, TensorFlow, Flower, Opacus, MONAI, scikit-learn et PySyft sont des logiciels libres sous licences respectives (BSD, Apache 2.0, MIT). Leur usage dans ce document est purement pédagogique et n'emporte aucune prescription commerciale.

Mentions relatives aux données et aux expérimentations

Document pédagogique à usage de formation interne. Les exemples de code fournis sont livrés à titre pédagogique ; les résultats numériques illustrés sont issus d'expérimentations conduites dans un cadre de recherche et ne constituent pas des performances de production.

Ce support synthétise les travaux de recherche du Département de Recherche & Développement de Pixel Software Design sur l'intelligence artificielle distribuée, l'apprentissage fédéré, la protection de la vie privée et les systèmes embarqués intelligents.

Sommaire

Structure détaillée du cours

Partie 1 — IA, Données et Systèmes Distribués 8

1.1 Apprentissage automatique : typologie et chaîne de production 9

1.2 Observabilité et supervision des systèmes 12

1.3 Calcul et analyse de graphes 18

1.4 Fouille de données 22

1.5 Systèmes Big Data et architectures distribuées 27

Annexes 40

A.1 Liste des abréviations 41

A.2 Table de notation 41

Partie	Thème	Contenu	Compétence visée	Durée
Partie 1	Fondations	Statistique, optimisation, courbes d'apprentissage	Comprendre et formaliser	2-3 semaines
Partie 2	Algorithmes FL	FedAvg, convergence, hétérogénéité, système	Implémenter et régler	3-4 semaines
Partie 3	Confidentialité	DP, attaques, défenses, agrégation sécurisée	Auditer et sécuriser	2-3 semaines
Partie 4	Applications	FedLLM, santé, V2X, FedRL, TinyML, graphes	Transposer à un domaine	3-4 semaines
Partie 5	Framework FedX	Architecture, API, benchmarks, exploitation	Industrialiser	2 semaines
Partie 6	Nouveaux fronts	FedCL, FedCI, SNN, FedGL, feuille de route	Ouvrir la recherche	2 semaines
Partie 7	TP et projets	9 labs, 5 projets, corrigés, évaluation	Faire et rendre	4-6 semaines

Préface

Comment utiliser ce cours

Ce support est conçu pour un parcours de formation et de recherche couvrant l'ensemble de la chaîne de valeur de l'intelligence artificielle distribuée : les fondations mathématiques, les algorithmes d'apprentissage fédéré, les menaces et défenses, les applications de pointe, une plateforme de référence opérationnelle, et enfin les nouveaux fronts ouverts par la recherche du Département.

Ce n'est pas un manuel de référence à lire d'un trait : c'est un **manuel opératoire**. Chaque chapitre alterne exposition, formalisation, mise en œuvre numérique et évaluation. Les listings sont complets et exécutables ; les tableaux comparatifs servent de grille de décision ; les schémas d'architecture servent de support de revue de conception.

Public et prérequis

Le contenu s'adresse à des étudiants de master et de doctorat, à des ingénieurs IA et à des praticiens de la recherche appliquée. Les prérequis sont la connaissance de l'apprentissage automatique classique, de l'algorithmique numérique et des bases de probabilités. Les parties 1 à 3 sont auto-suffisantes pour un étudiant qui découvre le domaine ; les parties 4 à 7 supposent la mécanique et les outils présentés dans la partie 2.

Conventions de lecture. Une expression mathématique est encadrée par deux signes dollars ; un identifiant, un fichier ou une commande est écrit en caractères à chasse fixe ; les encadrés *Définition*, *Théorème*, *Méthode* et *Attention* signalent les points à maîtriser et les pièges fréquents. Les renvois de la forme « voir chapitre 3.2 » désignent le second chapitre de la troisième partie.

Progression pédagogique

Apprentissage, observabilité, graphes, fouille et Big Data

Partie 1

IA, Données et Systèmes Distribués

Les cinq piliers de la chaîne de valeur numérique. On part du modèle qui apprend, on passe à la mesure du système qui le sert, puis aux structures qui relient les entités, aux méthodes qui extraient des connaissances, et enfin aux architectures qui rendent l'ensemble possible à l'échelle. Chaque module indiqué les outils de référence, mais l'accent est sur la décision de conception qui les sous-tend.

Objectifs de la partie

- Distinguer une tâche d'apprentissage d'un outil, et choisir l'un avant l'autre.
- Concevoir une chaîne d'observabilité en tenant compte du volume, de la rétention et du coût.
- Implémenter et comparer un plus court chemin, une centralité et une détection de communauté.
- Extraire des règles d'association et juger leur intérêt par le support et la confiance.
- Concevoir un traitement par lots ou en flux, et justifier ce choix en latence et en exactitude.

Plan de la partie

1. Apprentissage automatique : typologie et chaîne de production
2. Observabilité et supervision des systèmes
3. Calcul et analyse de graphes
4. Fouille de données
5. Systèmes Big Data et architectures distribuées

Chapitre 1.1

Apprentissage automatique : typologie et chaîne de production

Le choix de l'algorithme vient après le choix de la question. L'inverse est l'erreur la plus répandue.

La typologie : quatre questions, pas une liste d'outils

L'apprentissage automatique se classe selon la disponibilité des étiquettes, la nature de la sortie, et le moment où l'erreur est révélée. Ces trois axes suffisent à couvrir toutes les situations rencontrées en production.

Situation	Sortie attendue	Exemples d'algorithmesQuestion à poser d'abord	
Supervisé, sortie continue	un nombrerégression linéaire, forêt, réseau	quelle erreur coûte le plus cher ?	
Supervisé, sortie discrète	une classe	régression logistique, machines à vecteurs de support, arbre	quel est le coût d'un faux positif, d'un faux négatif ?
Non supervisé, structure	des groupes	k-moyens, densité, hiérarchique	le découpage obtenu est-il interprétable ?
Non supervisé, compression	moins de dimensions	analyse en composantes principales, voisinage	quelle variance faut-il vraiment garder ?
Renforcement	une politique d'actions	gradient de politique, recherche de politique	quelle récompense mesure-t-on réellement ?

Tableau 1 – Les situations d'apprentissage et l'algorithme correspondant

La dernière colonne est la seule qui compte vraiment. Un algorithme choisi avant la question conduit à un modèle impossible à évaluer correctement.

$$y = w^{\top} x + b,\ \hat{y} - y \Rightarrow w \leftarrow w - \eta \le ft(w^{\top} x - y) x$$

$$p(y = 1 \mid x) = \sigma \le ft(w^{\top} x + b),\ \sigma(z) = (1)/(1 + e^{-z}) \qquad \text{(D.1)}$$

La régression linéaire ajuste une droite et se corrigé par l'erreur résiduelle. La régression logistique ajuste une probabilité : la sigmoïde borne la sortie entre zéro et un, ce qui la rend utilisable comme score de confiance.

> **Astuce**
> La réduction de dimension et la segmentation partagent la même question sous-jacente : que perdre en échange de ce qu'on gagne ? L'analyse en composantes principales maximise la variance conservée et rend le résultat linéaire et lisible ; les méthodes de voisinage sont plus expressives et bien moins interprétables. Le choix se fait sur la nécessité ou non d'expliquer.

La chaîne de production d'un modèle

Algorithme 1 – De la donnée brute au modèle déployé

```
01  Recueillir, identifier la source, horodater chaque enregistrementContrôler la qualité :
valeurs manquantes, doublons, valeurs aberrantes
02  Corriger les erreurs de mesure connues par le métier
03  Séparer en trois jeux disjoints, avant toute transformation
04  Transformer : normalisation, encodage des catégories, réduction de dimension
05  Entraîner sur le jeu d'entraînement, ajuster sur le jeu de validation
06  Évaluer une seule fois sur le jeu de test, jamais avant
07  Rendre reproductible : version des données, du code, de la configuration
08  Déployer sous conteneur, surveiller, et prévoir le retour arrière
```

Entrées : Données brutes, question métier, contrainte de coût d'erreur • **Sorties :** Modèle versionné, jeu d'évaluation, rapport de métriques • **Complexité :** Effort dominant : la préparation, pas l'entraînement

La ligne 4 est irremplaçable et presque toujours faite trop tard. Séparer après avoir préparé les données laisse fuiter de l'information du test dans l'entraînement, et rend l'estimation optimiste.

```python
from dataclasses import dataclass
from typing import Sequence

import numpy as np
from sklearn.metrics import f1_score, precision_score, recall_score


@dataclass(frozen=True)
class RègleDePromotion:
"""Verdict de promotion d'une version de modele.

Une version n'est acceptee que si elle ameliore au moins une metrique
globale, si aucune metrique de sous-groupe ne se degrade au-dela du
seuil, et si elle respecte les contraintes de cout et de latence.

Le critere sur les sous-groupes est celui qu'on oublie le plus souvent.
Un modele peut gagner deux points de precision globale en degradant
severement un groupe de dix pour cent de la population : la moyenne
masque exactement ce qui doit etre surveille.
"""

gain_minimal: float = 0.005
perte_toleree: float = 0.01
latence_max_ms: int = 200

def accepter(self, candidat: dict, reference: dict,
sous_groupes: Sequence[str]) -> tuple[bool, dict]:
ecarts: dict = {}
accepte = True

for groupe in sous_groupes:
ecarts[groupe] = candidat[groupe] - reference[groupe]
if ecarts[groupe] < -self.perte_toleree:
accepte = False

gain = candidat["global"] - reference["global"]
ecarts["global"] = gain
if gain < self.gain_minimal:
accepte = False

if candidat.get("latence_ms", 0) > self.latence_max_ms:
accepte = False

return accepte, ecarts


def rapporter(y_true: np.ndarray, y_pred: np.ndarray,
seuil: float = 0.5) -> dict:
"""Transforme des scores continus en decisions, puis en metriques.

Le seuil de 0,5 n'a aucune signification en soit : il doit etre choisi
sur le jeu de validation, en fonction du cout relatif d'un faux
positif et d'un faux negatif. C'est la seule decision de ce module
qui demande un arbitrage metier, pas une mesure technique.
"""
decisions = (y_pred >= seuil).astype(int)
return {
"f1": f1_score(y_true, decisions),
"precision": precision_score(y_true, decisions),
"rappel": recall_score(y_true, decisions),
"taux_refus": float((y_pred < seuil * 0.6).mean()),
}
```

Listing 1 — Gouvernance d'un modèle : promotion et rapport de métriques

Exercice 1.1.1 (difficulte : Moyen)

Un modèle de crédit atteint 92 % de justesse sur l'ensemble de test, mais 71 % sur le plus petit groupe de population. Calculer l'exactitude non pondérée par groupe, puis l'exactitude par groupe, et en déduire la part de la population que la métrique globale cache. Que faut-il demander à l'équipe qui a produit ce chiffre ?

Indice : Un chiffre global n'est informatif que si l'on sait avec quelles populations il a été calculé.

Correction

- L'exactitude globale est une moyenne pondérée par la taille de chaque groupe. Si le groupe minoritaire pèse dix pour cent de la population, la contribution de ses vingt et un points d'écart au score global est deux points : exactement l'ordre de grandeur de l'écart annoncé. Il faut demander la décomposition complète par groupe et la taille de chaque groupe.

A retenir

- La question précède l'algorithme, et le coût de l'erreur décide de la métrique.
- La séparation des jeux de données précède toute transformation, sinon l'estimation est optimiste.
- Une métrique globale masque les sous-groupes : la décomposition fait partie de la mesure.
- Le seuil de décision est un arbitrage métier, pas un paramètre technique.

Chapitre 1.2

Observabilité et supervision des systèmes

Savoir si un système fonctionne est la moitié du travail. Savoir pourquoi il ne fonctionne pas est l'autre moitié.

La supervision historique consisted à interroger périodiquement un état : la machine est-elle allumée, la connexion est-elle établie. L'observabilité moderne répond à une question plus difficile : dans un système distribué, quelle est la cause de cet comportement observable ? Elle se construit sur trois signaux de nature et de coût très différents.

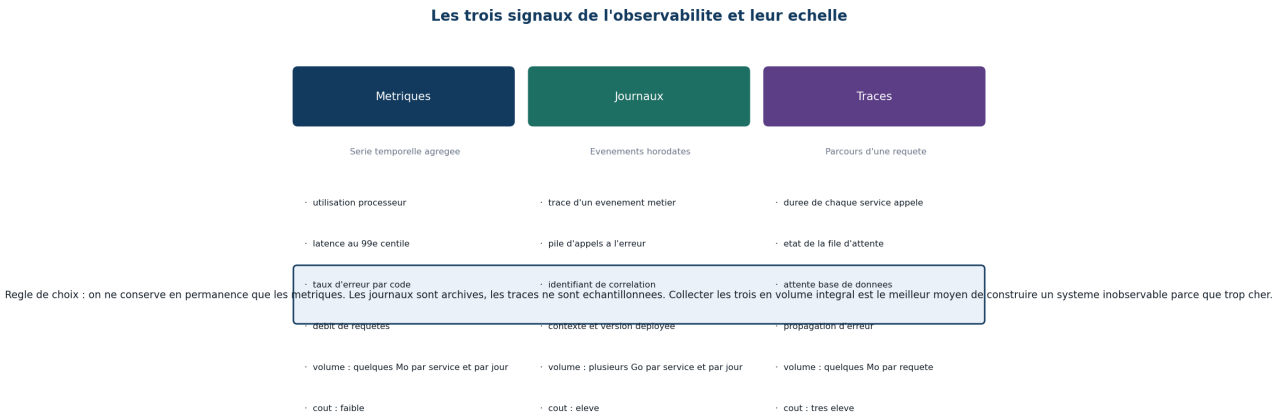


Figure 1 — Les trois signaux de l'observabilité et leur échelle

Signal	Question à laquelle il répond	GranularitéCoût de conservation	
Métrique	quel est le niveau et sa tendance ?une valeur par fenêtre de temps	faible, on garde tout	
Journal	que s'est-il passé à cet instant ?	un événement par ligne	élevé, on archive
Tracé	où le temps est-il passé ?	une portion par étape d'un parcours	très élevé, on échantillonne
Événement métier	l'activité a-t-elle un sens ?	un changement d'état métier	faible, très haute valeur
Profil de charge	où va la charge, et pour qui ?	une requête analysée	moyen, protection des données

Tableau 1 – Métriques, journaux, tracés : que répondre à quoi

La dernière ligne est la plus oubliée : un profil de charge répond à la seule question qui décide vraiment du budget, à savoir quel usage consomme la ressource.

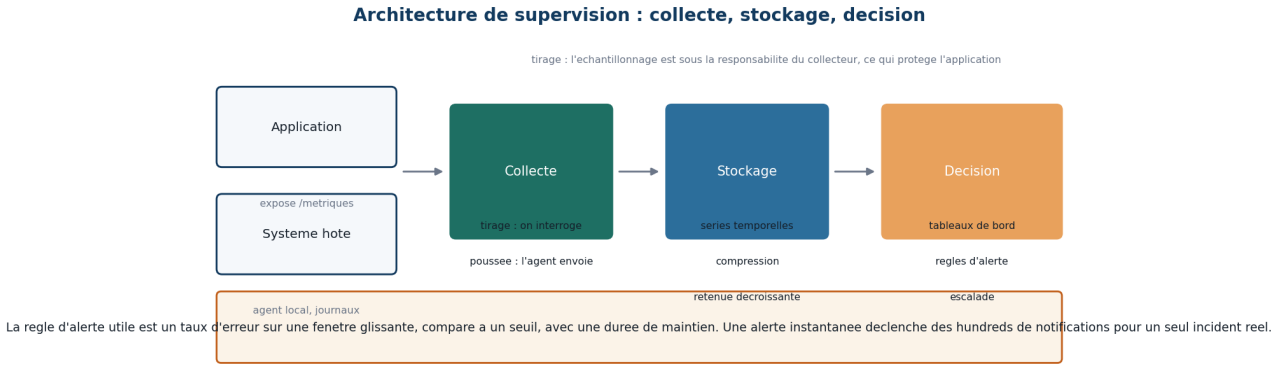


Figure 2 – Architecture de supervision : collecte, stockage, décision

Tirage ou poussée : une décision structurante

- **Tirage :** le collecteur interroge les cibles à intervalle fixe. L'application n'a rien à installer, et si elle tombe en panne le collecteur le constate immédiatement. C'est le mode par défaut.
- **Poussée :** l'agent envoie ses valeurs. Indispensable quand la cible est derrière un pare-feu ou hors du réseau d'administration, et pour les systèmes à très forte dispersion.
- **Court délai de scrape :** une fenêtre trop courte produit des mesures vides et du bruit ; une fenêtre trop longue allonge la détection d'incident. Trente secondes est un compromis usuel.
- **Conservation :** la rétention doit être dégressive — finesse pour les dernières heures, compression agressive au-delà.

Concevoir une règle d'alerte qui ne fatigue pas l'équipe

Une règle qui se déclenche sur un dépassement instantanée produit des centaines de notifications pour un seul incident. Les règles utilisables partagent trois propriétés :

- **Une condition de durée** : l'anomalie doit persister plus longtemps qu'un pic normal, par exemple deux minutes.

2. **Une fenêtre de calcul** : taux d'erreur sur cinq minutes, et non sur la dernière requête.
3. **Une action liée à la gravité** : avertissement pour surveillance, escalade vers un humain uniquement si la durée franchit un seuil.

La meilleure mesure de la qualité d'un système d'alerte est le nombre d'alertes ignorées par semaine : s'il est élevé, les règles sont trop nombreuses ou mal calibrées.

$$\text{taux}_{\text{erreur}} = (N_{\text{texténec}})(\Delta)N_{\square 0\square}(\Delta) \geq s \wedge \text{durée} \geq \tau \quad \text{(D.2)}$$

Les deux conditions sont indissociables. Un taux d'erreur élevé pendant une seconde est du bruit ; un taux faible maintenu deux minutes est une panne. C'est la conjonction qui distingue une alerte d'un signal.

Du symptôme à la cause : la méthode des quatre questions

- **Symptôme :** qu'est-ce qui est faux, vu de l'extérieur ? « Le service est lent pour 8 % des requêtes. »
- **Changement :** qu'est-ce qui a changé récemment ? Une version, une configuration, un volume, une dépendance.
- **Périmètre :** qui est affecté, et par quoi cela se distingue-t-il ? Un pays, un type de client, un modèle de requête.
- **Cause :** qu'est-ce qui explique simultanément le symptôme, le changement et le périmètre ?

La question la plus difficile est la quatrième, et c'est celle que l'observabilité bien conçue permet de trancher en quelques minutes au lieu de plusieurs heures. Un identifiant de corrélation propagé dans toutes les couches vaut souvent davantage que n'importe quel tableau de bord supplémentaire.

> **Surveillance d'un modèle et surveillance d'un service**
> Un service peut être parfaitement sain — latence normale, aucune erreur — et le modèle qu'il sert dégradé : les réponses sont plus courtes, plus vagues, ou silencieusement biaisées. La supervision technique ne voit rien. Il faut donc surveiller la qualité séparément : distribution des entrées, taux de refus, longueur des réponses, et surtout décomposition des métriques par groupe.

Travaux pratiques 1.1 — Superviser un service distribué de bout en bout

Duree : 3 h | Niveau : Intermédiaire | Outils : Prometheus pour la collecte, Grafana pour la visualisation, un jeu de requêtes reproduisant le trafic

Mettre en place une chaîne d'observabilité complète sur un service à trois composants, avec une règle d'alerte calibrée et un tableau de bord exploitable.

Deroulement

1. Exposer une métrique de latence par percentile et un compteur d'erreurs par code de retour dans une application à trois composants.
2. Configurer la collecte en mode tirage, puis vérifier la continuité des séries en cas d'arrêt d'un composant.
3. Écrire une règle d'alerte sur le taux d'erreur avec condition de durée, et déclencher une panne simulée pour vérifier qu'elle se déclenche une fois, pas plus.
4. Construire un tableau de bord en trois volets : santé technique, santé de la qualité, répartition de la charge par type de client.
5. Éteindre un composant pendant deux minutes et consigner le délai de détection, puis le délai de restauration.
6. Rédiger la règle d'escalade : qui est prévenu, par quel canal, à quelle heure.

Livrable attendu

- F
- i
- c
- h

- i
- e
- r
- d
- e
- c
- o
- n
- f
- i
- g
- u
- r
- a
- t
- i
- o
- n
- d
- u
- c
- o
- l
- l
- e
- c
- t
- e
- u
- r
- ,
- t
- a
- b
- l
- e
- a
- u
- d

- e
- b
- o
- r
- d
- ,
- r
- è
- g
- l
- e
- d
- '
- a
- l
- e
- r
- t
- e
- ,
- p
- r
- o
- c
- é
- d
- u
- r
- e
- d
- '
- e
- s
- c
- a
- l
- a
- d
- e

- ,
- e
- t
- r
- e
- l
- e
- v
- é
- d
- e
- s
- d
- é
- l
- a
- i
- s
- d
- e
- d
- é
- t
- e
- c
- t
- i
- o
- n
- e
- t
- d
- e
- r
- e
- s
- t
- a

- u
- r
- a
- t
- i
- o
- n
- .

Bareme indicatif

Critere	Points
L'alerte se déclenche exactement une fois sur la panne simulée	5
Le tableau de bord nomme le composant responsable en moins de deux minutes	3
La procédure d'escalade nomme un responsable et un délai	2

A retenir

- Métriques, journaux et tracés ont des coûts et des durées de conservation différents : on ne garde pas tout au même niveau.
- Une alerte utile est une conjonction de seuil et de durée, jamais un dépassement instantané.
- Le nombre d'alertes ignorées par semaine mesure la qualité du système d'alerte.
- La qualité d'un modèle se surveille séparément de la santé technique du service.

Chapitre 1.3

Calcul et analyse de graphes

Dès qu'une question porte sur des relations, la structure naturelle n'est pas la table mais le graphe.

Une base relationnelle répond vite à « quelles sont les lignes qui satisfont cette condition ». Elle répond mal à « quels sont les trois sauts qui séparent ces deux entités, et quel sommet se trouve sur tous les chemins possibles ». La deuxième question est une question de parcours dans un graphe, et elle est centrale en analyse de réseaux sociaux, en itinéraires, en détection de fraude et en recherche d'information.

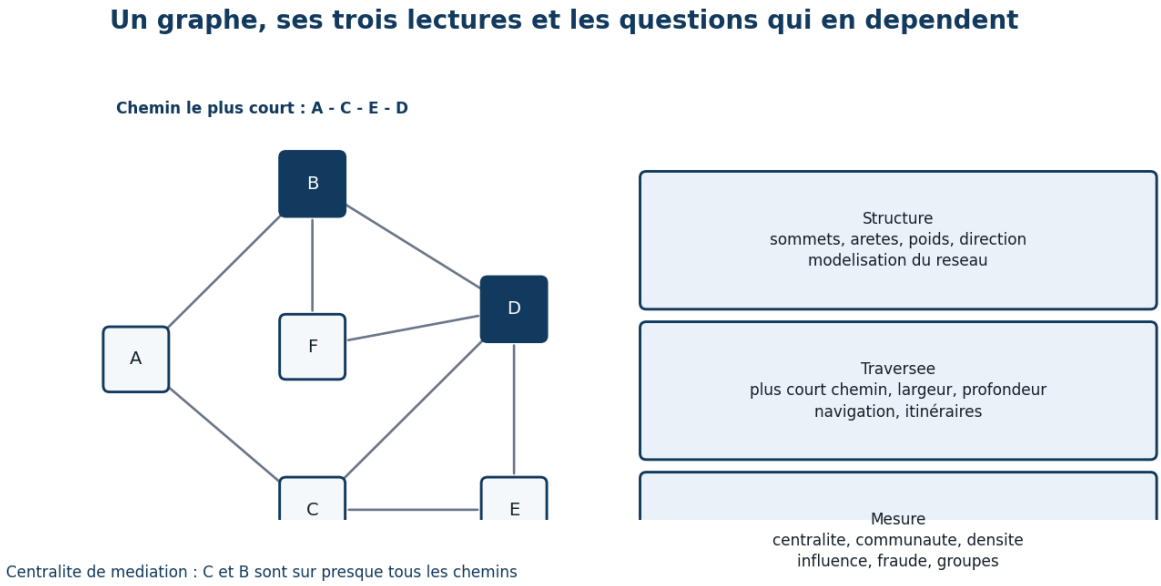


Figure 1 – Un graphe, ses trois lectures et les questions qui en dépendent

$$G = (V, E),\ E \subseteq V \times V,\ \deg^{+}(v) = \leq ft|\ (v, u) \in E\ |$$

$$PR(v) = (1 - \alpha)(1)/(N) + \alpha \sum_{u \to v}(PR(u))/(\deg^{+}(u)) \qquad (D.3)$$

La première expression fixe le vocabulaire : sommets, arêtes, degré sortant. La seconde est l'équation de PageRank : un sommets vaut une part constante, plus ce que ses voisins lui transmettent au prorata de leur propre nombre de liens.

Le plus court chemin : deux algorithmes, deux usages

	Dijkstra	A*
Principe	on explore par coût croissant du cheminon explore par coût estimé du chemin complet	
Fonction de coût	poids réels des arêtes	poids réels plus une estimation du coût restant
Condition de validité	poids positifs ou nuls	estimation admissible : jamais supérieure au coût réel
Garanti optimal	toujours	seulement si l'estimation est admissible
Coût	explore tout le voisinage atteint	dirigé, explore beaucoup moins
Usage	plus court chemin depuis une source unique	navigation, planification, jeux

Tableau 1 – Dijkstra et A* : le même principe, deux conditions

L'estimation admissible n'a pas besoin d'être précise : elle n'a pas besoin de sous-estimer le coût restant. Une estimation qui surestime transforme A* en recherche gloutonne : plus rapide, mais elle peut renvoyer une solution dégradée sans le signaler.

```python
import heapq
from collections import defaultdict
from typing import Callable, Iterable


def plus_court_chemin(graphe: dict[str, list[tuple[str, float]]],
depart: str, arrivee: str) -> tuple[float, list[str]]:
"""Recherche gloutonne avec file de priorite.

La file de priorite impose l'ordre d'exploration : le sommet traite
est toujours celui dont le cout cumule est le plus faible. Ce detail
est la seule difference avec une file d'attente ordinaire, et c'est
lui qui fait passer la complexite de quadratique a quasi lineaire.
"""
if depart not in graphe:
return float("inf"), []

couts: dict[str, float] = {depart: 0.0}
precedents: dict[str, str] = {}
vus: set[str] = set()
file: list[tuple[float, str]] = [(0.0, depart)]

while file:
cout, sommet = heapq.heappop(file)
if sommet in vus:
continue
vus.add(sommet)

if sommet == arrivee:
chemin = [arrivee]
while chemin[-1] != depart:
chemin.append(precedents[chemin[-1]])
return cout, list(reversed(chemin))

for voisin, poids in graphe.get(sommet, ()):
nouveau = cout + poids
if nouveau < couts.get(voisin, float("inf")):
couts[voisin] = nouveau
precedents[voisin] = sommet
heapq.heappush(file, (nouveau, voisin))

return float("inf"), []


```

Listing 1 — Plus court chemin : Dijkstra et A* dans un même module

```python
def plus_court_chemin_cible(
graphe: dict[str, list[tuple[str, float]]],
distance: Callable[[str, str], float],
depart: str, arrivee: str) -> tuple[float, list[str]]:
"""Meme algorithme, avec estimation admissible du cout restant."""
couts = {depart: 0.0}
precedents: dict[str, str] = {}
vus: set[str] = set()
file: list[tuple[float, float, str]] = [
(distance(depart, arrivee), 0.0, depart)]

while file:
_, cout, sommet = heapq.heappop(file)
if sommet in vus:
continue
vus.add(sommet)
if sommet == arrivee:
chemin = [arrivee]
while chemin[-1] != depart:
chemin.append(precedents[chemin[-1]])
```

```
    return cout, list(reversed(chemin))

for voisin, poids in graphe.get(sommet, ()):
    nouveau = cout + poids
    if nouveau < couts.get(voisin, float("inf")):
        couts[voisin] = nouveau
        precedents[voisin] = sommet
        priorite = nouveau + distance(voisin, arrivee)
        heapq.heappush(file, (priorite, nouveau, voisin))

return float("inf"), []
```

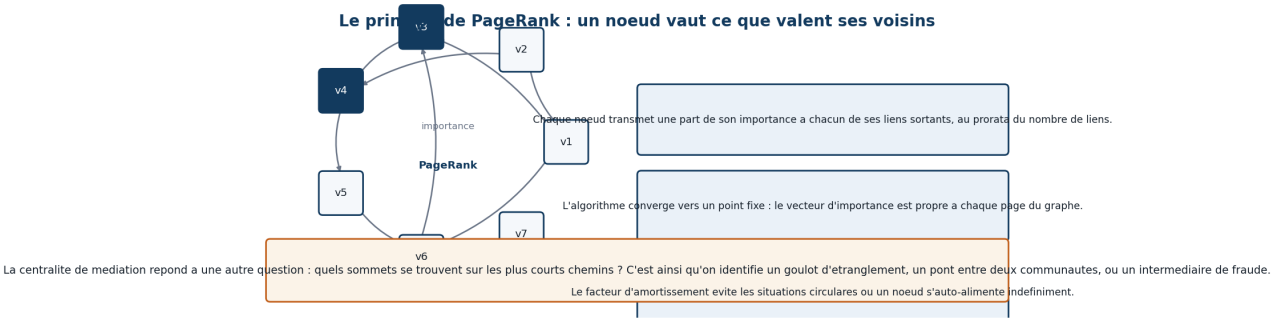


Figure 2 – PageRank : un sommet vaut ce que valent ses voisins

Les mesures de centralité et ce qu'elles/signalent

Mesure	Question à laquelle elle répond	InterprétationDomaine d'usage	
Degré	combien de relations directes ?popularité brute, facilement trompeuse	réseaux sociaux, epidemiology	
PageRank	combien de reputable neighbours ?	influence propagée par le graphe orienté	recherche, influence
Médiation	sur quels chemins ce sommet est-il indispensable ?	goulot d'étranglement, pont entre communautés	analyse de réseau, fraude
Proximité	combien de chemins courts vers tous les autres ?	position périphérique ou centrale	diagnostic de réseau
Communauté	quels sommets se ressemblent structurellement ?	groupes denses à l'intérieur, clairs entre eux	segmentation, détection de fraude coordonnée

Tableau 2 – Quatre mesures, quatre questions

Le degré n'est presque jamais la bonne mesure. Un sommet très connecté peut n'être connu que d'un seul groupe. Ce qui compte, c'est la qualité des voisins, pas leur nombre.

Un compte suivie par cent mille personnes sans activité, et relayé par elles, a un degré élevé et une influence nulle. La mesure juste regarde vers qui pointe chaque arête, et non seulement combien il y en a. Confondre les deux conduit à recommander des éléments que personne ne consumera réellement.

Le piège du degré comme mesure d'influence

Astuce

Pour la détection de fraude en transactions, la mesure utile est presque toujours la médiation plutôt que le degré : un compte intermédiaire qui apparaît sur les plus courts chemins entre plusieurs comptes est bien plus suspect qu'un compte très actif.

A retenir

- Les questions de parcours et de relations exigent une structure de graphe, pas des jointures récursives.
- A* est optimal si et seulement si son estimation du coût restant est admissible.
- Le PageRank mesure une influence propagée, pas un nombre de relations.
- La médiation signale les sommets indispensables aux parcours, donc les goulots d'étranglement.

Chapitre 1.4

Fouille de données

Extraire des connaissances ne signifie pas trouver des corrélations : cela signifie en trouver qui resistent à une vérification.

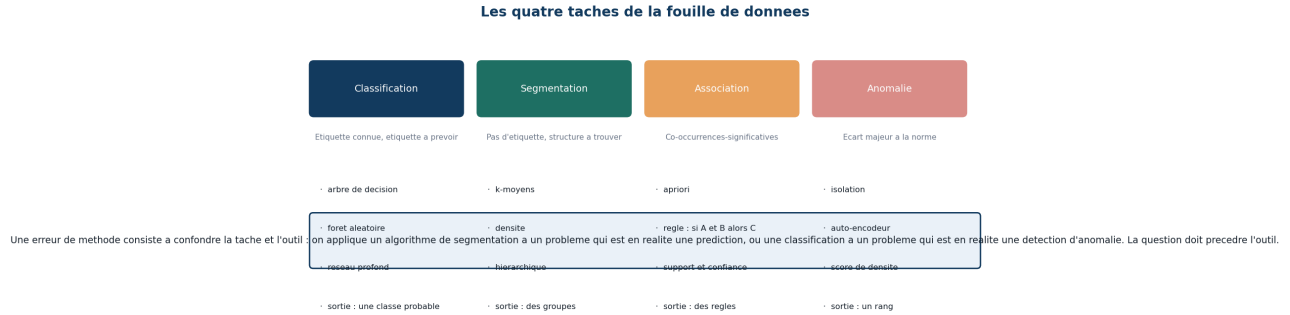


Figure 1 – Les quatre tâches de la fouille de données

Classification : l'arbre comme langage de décision

Un arbre de décision est une suite de questions binaries. Chaque nœud pose une question sur une variable, chaque feuille porte une prédiction. Le critère de choix de la question de premier rang est la réduction d'incertitude qu'elle procure.

$$\text{Entropie}(S) = -\sum_{c \in C} (|S_c|)/(|S|) \log_2 (|S_c|)/(|S|)$$
$$\text{Gain}(S, A) = \text{Entropie}(S) - \sum_{v} (|S_v|)/(|S|) \text{Entropie}(S_v) \quad \text{(D.4)}$$

L'entropie mesure l'hétérogénéité des étiquettes. Le gain est la réduction obtenue en séparant les données selon la variable testée. L'arbre retient la variable de gain maximal, et recommence sur chaque partition.

Surapprentissage, là où presque tout se joue

Un arbre sans contrainte memorise chaque feuille d'entraînement et donne une exactitude parfaite en validation sur les mêmes données, puis une exactitude médiocre sur des données nouvelles. Les trois coupes principales sont : limiter la profondeur, imposer un nombre minimal d'exemples par feuille, et élaguer les branches dont le gain est inférieur à un seuil. La forêt aléatoire va plus loin : elle construit des arbres sur des tirages aléatoires des variables et des exemples, puis moyenne leurs décisions. La dispersion d'un seul arbre devient une estimation de l'incertitude, ce qui est souvent la seule chose qui manque à une décision.

Segmentation : deux philosophies

	K-moyens	Densité
Principe	minimise la dispersion autour de centroïdesregroupe les zones denses séparées par du vide	
Nombre de groupes	fixé à l'avance	découvert, non paramétrique
Forme des groupes	sphérique, de taille voisine	arbitraire, taille inégale
Bruit	assigné au groupe le plus proche	peut être identifié et exclu
Valeur d'initialisation	critique, plusieurs essais	peu importante
Choix du paramètre	nombre de groupes	rayon de voisinage et seuil
Limite principale	impose une forme aux données réelles	sensible aux échelles de densité

Tableau 1 – K-moyens et densité : ce qui les sépare

K-moyens suppose que les groupes sont compacts. Sur des données réelles en orbites concentriques, il échoue complètement : la densité est alors le bon choix, parce qu'elle n'impose aucune forme.

Apriori : du treillis d'items frequents a une regle

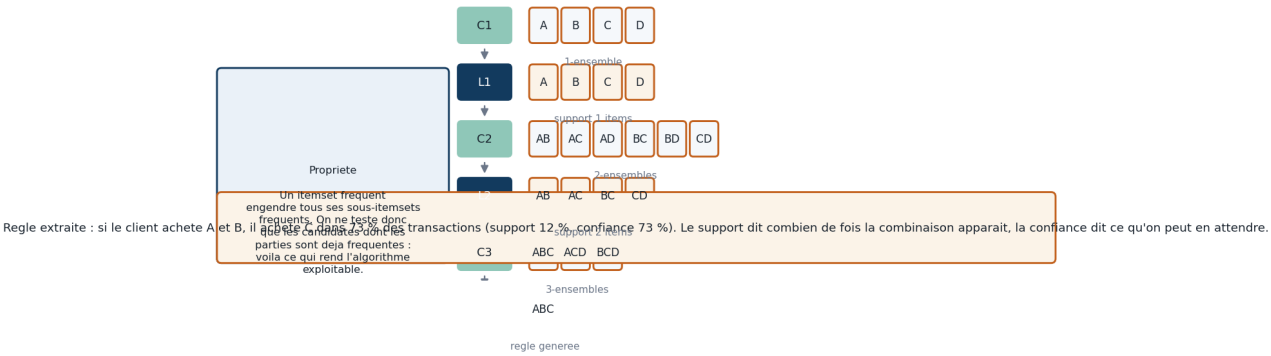


Figure 2 – Apriori : du treillis d'items fréquents à une règle

$$\text{support}(A \to B) = (n(A \cup B))/(N),\ \text{confiance}(A \to B) = (n(A \cup B))/(n(A)) \quad \text{(D.5)}$$

Le support mesure la fréquence de la co-occurrence, la confiance mesure la probabilité conditionnelle. Une règle à forte confiance et faible support décrit une co-occurrence rare mais parfaitement fiable – ce n'est pas pour autant une rête exploitable.

Juger une règle d'association, dans l'ordre

- **Support :** combien de transactions ? En dessous de la centaine, la règle n'a aucun usage statistique.

2. **Confiance :** la consequent est-il réellement tiré par l'antécédent, ou seulement par sa fréquence propre ?
3. **Levier :** la confiance exceede-t-elle nettement la fréquence de la consequent seule ? C'est le seul test qui distingue une vraie règle d'un artefact de fréquence.
4. **Négativité :** le nombre de règles est tsunami : cent transactions en produisent des milliers. Il faut un critère de sélection final, sinon personne ne lira le résultat.
5. **Décision :** une règle n'est un résultat que si elle déclenche une action mesurable et évaluable.

Détection d'anomalies : deux approches, deux usages

	Approche par densité	Approche par reconstruction
Principe	mesure la rareté d'un point dans son voisinagereconstruit l'entrée et mesure l'écart	
Exemple	isolation des arbres	auto-encodeur
Exigence	une notion de voisinage	des données d'entraînement représentatives
Force	simple, explicable, détecte l'inconnu	capte des anomalies de forme complexe
Faiblesse	sensible à l'échelle et à la densité	une anomalie fréquente dans l'entraînement devient normale
Usage	détection de fraude bancaire, intrusion	contrôle industriel, maintenance prédictive

Tableau 2 – Anomalies par densité et par reconstruction

La faiblesse de l'auto-encodeur est un problème de données, pas de méthode : ce qui était rare à l'entraînement devient la norme, et le modèle l'apprendra comme tel.

Un patient en état critique et une panne machine produisent la même signature statistique. Statistiquement, la détecter est aisé ; deciding qu'elle doit déclencher une alerte relève du métier. Sans cette règle métier écrite, un système de détection d'anomalies produit un flux d'alertes que personne ne traité, ce qui est pire que pas de système du tout.

L'anomalie qui n'est pas une erreur

Travaux pratiques 1.2 – Panier de marché et détection d'anomalies

Duree : 4 h | Niveau : Avance | Outils : Un jeu de transactions en trois colonnes : identifiant de panier, article, quantité

Extraire des règles d'association exploitables d'un panier réel, puis construire un score d'anomalie reproductible sur les transactions d'un client.

Deroulement

1. Écrire le générateur d'itemsets fréquents par largeur croissante, en éliminant les candidats dont un sous-itemset est infrequent.
2. Mesurer le support, la confiance et le levier de toutes les règles extraites, puis justifier le critère qui retient les trois meilleures.
3. Afficher la courbe de nombre de règles en fonction du seuil de support, et expliquer la forme observée.
4. Construire un score d'anomalie fondé sur la rareté locale, puis l'appliquer aux transactions d'un même client et commenter les cinq plus fortes anomalies.
5. Répondre par écrit : parmi ces anomalies, lesquelles justifient une enquête, et sur quel critère métier ?

Livrable attendu

- M
- o
- d
- u
- l
- e
- P
- y
- t

- h
- o
- n
- ,
- t
- a
- b
- l
- e
- a
- u
- d
- e
- s
- r
- è
- g
- l
- e
- s
- a
- v
- e
- c
- l
- e
- u
- r
- s
- t
- r
- o
- i
- s
- m
- e
- s
- u

- r
- e
- s
- ,
- c
- o
- u
- r
- b
- e
- d
- e
- s
- é
- l
- e
- c
- t
- i
- o
- n
- ,
- n
- o
- t
- e
- d
- '
- a
- n
- a
- l
- y
- s
- e
- s
- u
- r

- l
- e
- s
- a
- n
- o
- m
- a
- l
- i
- e
- s
- d
- é
- t
- e
- c
- t
- é
- e
- s
- .

Bareme indicatif

Critere	Points
La règle est justifiée par le levier, non par la seule confiance	4
Le nombre de règles en fonction du seuil est expliqué	2
L'analyse distingue ce qui est statistique et ce qui est décisionnel	4

A retenir

- Un arbre se construit par gain maximal d'information et se contraint par élagage, profondeur minimale et taille de feuille.
- La dispersion d'une forêt aléatoire fournit une estimation d'incertitude qu'un arbre seul ne donné pas.
- K-moyens impose une forme aux groupes, la densité ne l'impose pas : le choix suit les données.
- Une règle d'association n'est exploitable qu'après le test du levier et un critère de sélection explicite.

Chapitre 1.5

Systèmes Big Data et architectures distribuées

Distribuer n'est pas une optimisation de performance : c'est la condition pour accepter l'échelle,

avec le prix de la cohérence.

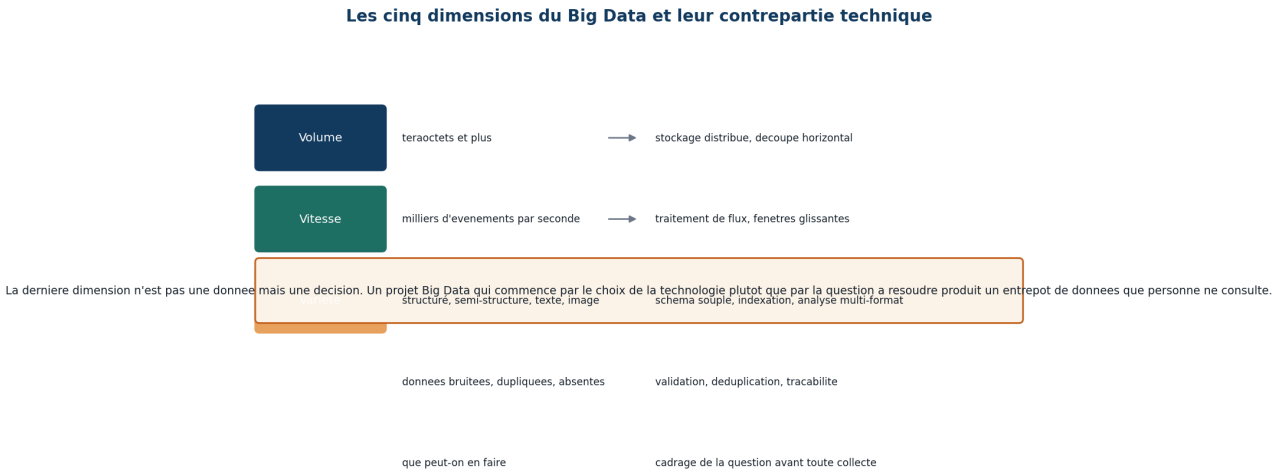


Figure 1 – Les cinq dimensions du Big Data et leur contrepartie technique

Pourquoi distribuer, et à quel prix

On distribue lorsque la donnée ne tient pas sur une machine, ou lorsque le débit d'écriture dépasse ce qu'un seul disque n'encaisse. La distribution résout ces deux problèmes et en introduit un troisième : la cohérence. Dans un système à plusieurs machines, deux lectures consécutives peuvent voir des états différents, et toute transaction qui traverse plusieurs nœuds doit être coordonnée.

Acquis	Coût correspondingly
Capacité de stockage extensiblela donnée est découpée, l'accès doit connaître la découpe	
Débit d'écriture et de lecture	la charge doit être répartie sans déséquilibre
Tolérance à la panne	il faut détecter et reconstruire, et le diagnostic devient plus difficile
Extensibilité linéaire	la coordination coûte, et son coût croît avec le nombre de machines
Isolation des charges	le partage de ressources crée des interférences imprévisibles

Tableau 1 – Ce que la distribution achète et ce qu'elle coûte

La dernière ligne est la plus sous-estimée. Sur un cluster partagé, une requête lourde peut dégrader toutes les autres sans qu'aucune métrique ne le signale individuellement.

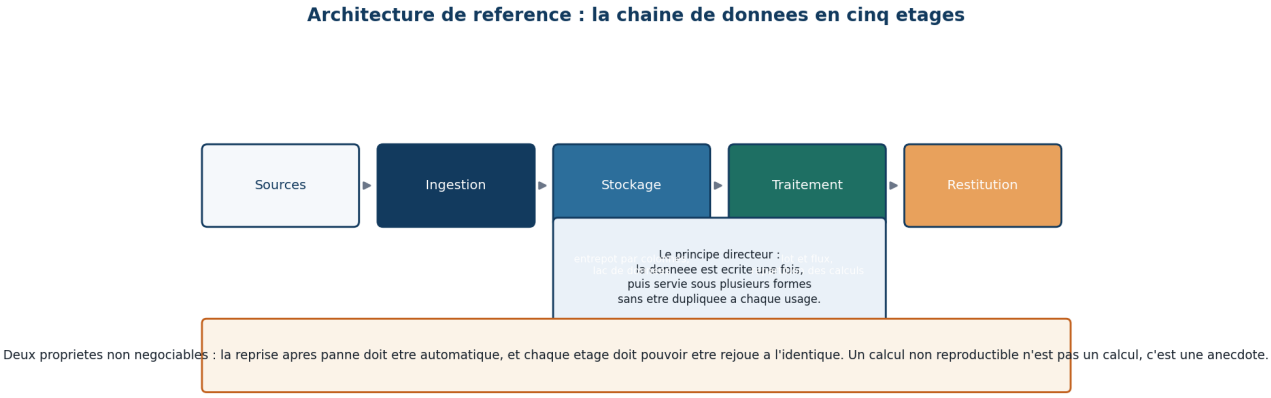


Figure 2 – Architecture de référence : la chaîne de données en cinq étages

Stockage : pourquoi la disposition en colonnes change tout

- **Stockage par lignes :** une ligne complète est l'unité lue. Bon pour les mises à jour ponctuelles, coûteux pour l'analyse, qui lit quelques colonnes de millions de lignes.
- **Stockage par colonnes :** seule la colonne demandée est lue, et les valeurs d'une même colonne se compressent très bien. C'est la disposition qui rend praticable l'analyse sur de grands volumes.
- **Indexation :** l'index, les listes triées, accélère la sélection mais ralentit l'écriture. Sur un système à forte écriture, c'est un arbitrage permanent.
- **Partage :** répartir par plage de clés met les écritures réparties, mais rend les parcours multi-clés plus coûteux.

Traitement : deux philosophies

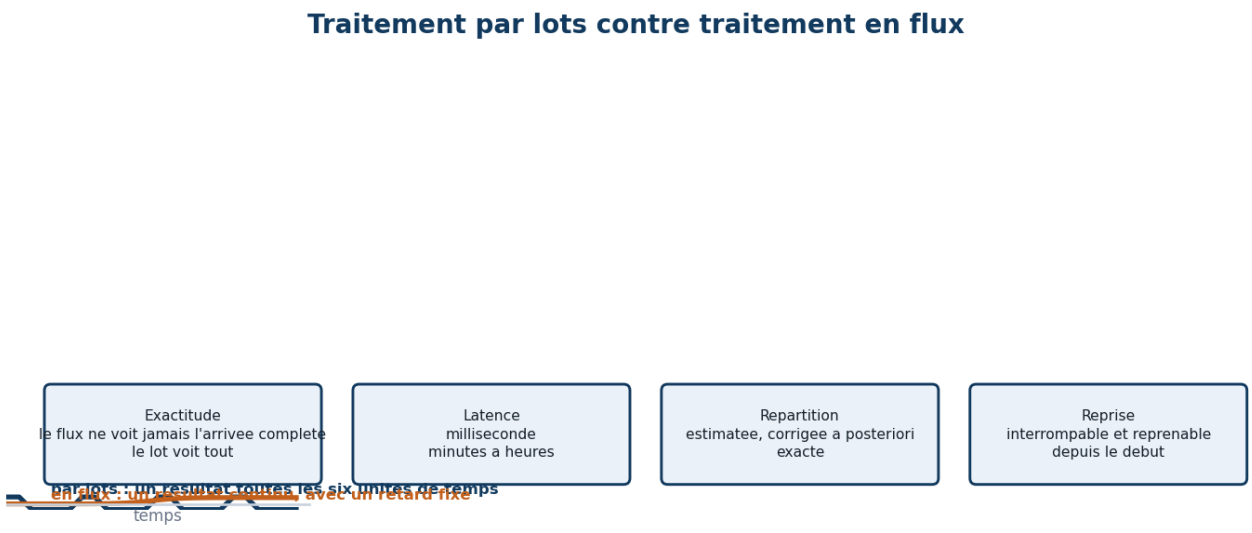


Figure 3 – Traitement par lots contre traitement en flux

Critère	Par lots	En flux	Choix pratique
Latence	minutes à heures	millisecondesle flux si l'action doit suivre l'événement	
Exactitude	voit l'intégralité de la période	estimate, corrigée ensuite	le lot si le calcul doit être exact
Reprise	relance complète	reprise au point de reprise	flux, si l'état est externalisé
Coût	bon pour de gros volumes	coût par unité plus élevé	lots pour l'historique, flux pour le direct
Complexité	faible	gestion d'état, temps d'événement	commencer par lots

Tableau 2 – Par lots, en flux, ou les deux

En pratique, les deux coexistent presque toujours : flux pour l'alerte immédiate, lots pour la consolidation exacte. C'est l'architecture de référence, pas le choix binaire.

Traiter un flux : fenêtre, temps et état

Trois notions à décider avant d'écrire la première ligne :

- **Fenêtre** : par temps fixe (une minute) ou par nombre d'événements (les mille premiers) ? La première donne un temps constant, la seconde un volume constant.

2. **Temps d'événement** : l'horodatage du capteur, qui peut être en retard, ou l'horodatage de réception, qui est toujours juste. Le premier est plus fidèle, le second plus simple.
3. **État** : les traitements inhomogènes supposent de conserver un état au-delà d'un événement. Cet état doit être periodically sauvegardé, sinon une panne le perd.

L'erreur la plus courante consiste à traiter un flux comme un lot : sans fenêtre, chaque événement est traité seul et aucun agrégat n'est possible.

Astuce

Quand vous ne savez pas si un calcul doit être exact ou approximé, la réponse est de faire les deux : l'un en flux pour l'alerte, l'autre en lots pour la consolidation. Les deux résultats doivent se rejoindre à une marge près ; l'écart entre eux est lui-même un excellent signal de supervision.

Cadre juridique de l'usage des données

Le traitement de données à grande échelle est encadré, en particulier pour les données personnelles : finalité déterminée, base légale explicite, minimisation des données collectées, information des personnes concernées, et droit d'opposition et d'effacement. Ces principes sont indépendants de la technique : un entrepôt distribué n'améliore ni n'aggrave la conformité, il rend simplement l'incapacité à répondre à une demande de suppression beaucoup plus coûteuse — parce qu'il faut retrouver toutes les copies. C'est le premier argument à opposer avant de distribuer une donnée.

Projet – Concevoir une chaîne complète, de la source à la restitution

Duree : 6 semaines | Equipe : 3 a 5 etudiants | Depot : gitlab + rapport

Une entreprise de logistique veut suivre ses véhicules, détecter les anomalies de conduite et produire un tableau de bord temps réel, tout en conservant un historique exact pour la facturation. Le volume annuel est de l'ordre de deux milliards d'enregistrements.

Missions

1. Spécifier le schéma des événements, en distinguant ce qui est stocké, ce qui est agrégé et ce qui est jeté.
2. Choisir le découpage du stockage, en justifiant le critère de répartition et ses conséquences sur les requêtes d'intervalle.
3. Définir la fenêtre de temps et le temps d'événement du traitement de flux, puis écrire la règle d'alerte sur la détection d'anomalie de conduite.
4. Concevoir la consolidation exacte par lots, et indiquer comment elle se réconcilie avec l'alerte de flux.

5. Écrire le plan de reprise après panne : ce qui est reconstruit, en combien de temps, et ce qui est perdu dans l'intervalle.
6. Établir ce qu'il faut pour répondre à une demande d'effacement d'un conducteur, en traversant toute la chaîne.

Livrables

- D
- o
- c
- u
- m
- e
- n
- t
- d
- '
- a
- r
- c
- h
- i
- t
- e
- c
- t
- u
- r
- e
- e
- n
- c
- i
- n
- q
- é
- t
- a
- g
- e

- s
- ,
- s
- c
- h
- é
- m
- a
- d
- e
- d
- é
- c
- o
- u
- p
- a
- g
- e
- ,
- p
- l
- a
- n
- d
- e
- r
- e
- p
- r
- i
- s
- e
- a
- p
- r
- è
- s

- p
- a
- n
- n
- e
- ,
- e
- t
- n
- o
- t
- e
- s
- u
- r
- l
- a
- c
- o
- n
- f
- o
- r
- m
- i
- t
- é
- a
- u
- x
- d
- e
- m
- a
- n
- d
- e
- s

- d
- '
- e
- f
- f
- a
- c
- e
- m
- e
- n
- t
- .

Évaluation

- L
- e
- j
- u
- r
- y
- v
- é
- r
- i
- f
- i
- e
- t
- r
- o
- i
- s
- p
- o
- i
- n
- t
- s

- :
- l
- e
- c
- r
- i
- t
- è
- r
- e
- d
- e
- d
- é
- c
- o
- u
- p
- a
- g
- e
- e
- s
- t
- j
- u
- s
- t
- i
- f
- i
- é
- p
- a
- r
- l
- e
- t

- y
- p
- e
- d
- e
- r
- e
- q
- u
- ê
- t
- e
- ,
- l
- a
- r
- é
- c
- o
- n
- c
- i
- l
- i
- a
- t
- i
- o
- n
- e
- n
- t
- r
- e
- f
- l
- u
- x

- e
- t
- l
- o
- t
- s
- e
- s
- t
- e
- x
- p
- l
- i
- c
- i
- t
- e
- ,
- e
- t
- l
- '
- e
- f
- f
- a
- c
- e
- m
- e
- n
- t
- e
- s
- t
- t
- r

- a
- i
- t
- é
- c
- o
- m
- m
- e
- u
- n
- e
- c
- o
- n
- t
- r
- a
- i
- n
- t
- e
- d
- e
- c
- o
- n
- c
- e
- p
- t
- i
- o
- n
- e
- t
- n
- o

- n
- c
- o
- m
- m
- e
- u
- n
- v
- e
- r
- r
- o
- u
- .

A retenir

- La distribution achète de l'échelle et introduit de la coordination : le prix est à chiffrer avant.
- La disposition en colonnes rend praticable l'analyse, mais coûte à l'écriture.
- Par lots et en flux coexistent : flux pour l'alerte, lots pour la consolidation exacte.
- Une fenêtre, un temps d'événement et un état : trois décisions précèdent toute ligne de traitement de flux.
- L'effacement d'une donnée se paie en conception, pas en procédure d'urgence.

Abréviations, notation et ressources de consultation

Annexes

Annexes

Ce cours se termine par deux annexes de référence. La première donne la signification de chaque sigle employé, sous les deux formes — sigle et développé — car la litterature emploie indifféremment l'une ou l'autre. La seconde donne la signification de chaque symbole mathématique : elle permet de relire une démonstration sans revenir au chapitre dont elle est extraite, et de vérifier qu'un même symbole garde le même sens d'un chapitre à l'autre.

Chapitre A.1

Liste des abréviations

Les sigles ci-dessous sont ceux que le cours emploie. Chacun est donné avec sa forme développée et le sens précis qu'il prend ici, qui n'est pas toujours celui qu'on lui donne ailleurs.

Un sigle employé sans être développé oblige le lecteur à le reconnaître au passage, ou à l'abandonner. Le tableau ci-dessous ferme la liste de ceux que le cours emploie : il est exhaustif pour ce document, et non pour la discipline.

Sigle	Forme développée	Sens dans ce cours
ACID	Atomicity, Consistency, Isolation, Durability	Propriétés qu'une transaction doit satisfaire dans une base de données.
CAP	Consistency, Availability, Partition tolerance	Trio de propriétés dont on ne peut garantir que deux à la fois.
CRUD	Create, Read, Update, Delete	Quatre opérations de base sur une ressource.
ETL	Extract, Transform, Load	Chaîne d'alimentation d'un entrepôt de données.
OLAP	Online Analytical Processing	Traitement analytique en ligne, orienté agrégation.
OLTP	Online Transaction Processing	Traitement transactionnel en ligne, orienté écriture.
ORM	Object-Relational Mapping	Couche d'accès qui relie les objets du programme aux tables relationnelles.
PR	Pull Request	Proposition de fusion d'une branche, soumise à la revue.
RAID	Redundant Array of Inexpensive Disks	Agrégation de disques en bandes ou en miroirs.
RPO	Recovery Point Objective	Perte de données maximale tolérée après un incident, exprimée en durée.
RTO	Recovery Time Objective	Délai maximal de reprise d'activité après un incident.
SaaS	Software as a Service	Logiciel fourni hébergé, par opposition à une installation locale.
SGBD	Système de gestion de base de données	Logiciel de stockage et d'interrogation des données structurées.
sharding	Découpage horizontal	Répartition des lignes d'une table entre plusieurs instances.

Tableau 1 – Abréviations employées dans le cours

Chapitre A.2

Table de notation

Les symboles ci-dessous gardent le même sens d'un chapitre à l'autre. Lorsqu'un chapitre doit réemployer un symbole pour un autre objet, il le redéclare sur la page même.

La notation est celle du cours : vecteurs en gras, matrices en majuscules grasses, scalaires en italique. Les espérances sont notées E, les lois de données D, les espaces de paramètres Θ.

Symbole	Signification
D	Distribution des données.
n	Nombre total d'enregistrements.
K	Nombre de partitions, de fragments ou de cassettes.
k	Index d'une partition.
n_k	Nombre d'enregistrements de la partition k.
$p_k = (n_k)/(n)$	Part de la partition k dans le total.
T	Nombre de tranches, de tours ou de répliques.
t	Indice de tour ou de période.
x_i	Le i-ème enregistrement.
y_i	Étiquette associée à x_i.
μ	Moyenne empirique d'une grandeur.
σ^2	Variance empirique d'une grandeur.
$h(x)$	Fonction de hachage, qui répartit une clé sur les partitions.
$H(X)$	Entropie de Shannon, en bits si le logarithme est en base 2.
C_{ij}	Matrice de cooccurrence entre les colonnes i et j.
λ_j	Valeur propre j d'une matrice, donc variance expliquée par l'axe correspondant.

Tableau 1 – Symboles mathématiques et leur signification

Convention

Toute divergence constatée entre cette table et un chapitre signale une erreur de transcription, non une définition nouvelle : la table fait foi.

A retenir

- Un sigle employé sans être listé ici est un oubli de l'auteur, non une définition nouvelle.
- Un symbole qui change de sens d'un chapitre à l'autre rend toute démonstration ininterprétable : c'est la principale cause des contresens que nous avons relevés.
- Ces deux tables sont les seules parties du cours à maintenir à la main : le reste se relit tout seul.

Index des figures et des tableaux

Référence rapide du matériel visuel

Figures

1. Les trois signaux de l'observabilité et leur échelle
2. Architecture de supervision : collecte, stockage, décision
1. Un graphe, ses trois lectures et les questions qui en dépendent
2. PageRank : un sommet vaut ce que valent ses voisins
1. Les quatre tâches de la fouille de données
2. Apriori : du treillis d'items fréquents à une règle
1. Les cinq dimensions du Big Data et leur contrepartie technique
2. Architecture de référence : la chaîne de données en cinq étages
3. Traitement par lots contre traitement en flux

Tableaux

1. Les situations d'apprentissage et l'algorithme correspondant
1. Métriques, journaux, tracés : que répondre à quoi
1. Dijkstra et A* : le même principe, deux conditions
2. Quatre mesures, quatre questions
1. K-moyens et densité : ce qui les sépare
2. Anomalies par densité et par reconstruction
1. Ce que la distribution achète et ce qu'elle coûte
2. Par lots, en flux, ou les deux
1. Abréviations employées dans le cours
1. Symboles mathématiques et leur signification

Fin du document

Intelligence Artificielle Avancée & Federated Learning — Fondations mathématiques de l'apprentissage automatique, algorithmes fédérés, protection de la vie privée, défense contre les attaques, grands modèles de langage fédérés, systèmes embarqués, renforcement appris, applications critiques (santé, V2X), plateforme de référence FedX et feuille de route de recherche.
Ghiloufi Jamel, Architecte Logiciel Principal — Pixel Software Design
Département de Recherche & Développement — Recherche
Pixel Software Design — Édition 2026

© 2026 Pixel Software Design. Tous droits réservés.

Ce document est la propriété exclusive de Pixel Software Design. Toute reproduction, représentation, adaptation ou diffusion, totale ou partielle, par quelque procédé que ce soit et sur quelque support que ce soit, est interdite sans autorisation écrite préalable de l'auteur.

Document généré automatiquement le 29/09/2026 – mise en page Pixel Software Design.